

Transaccionalidad con Spring



✿ Lección: Agregar la Capa de Servicio y Manejo Transaccional en Spring Boot

📁 Introducción

En esta lección vamos a completar nuestra arquitectura implementando la **capa de servicio** en nuestra aplicación de Spring Boot con Thymeleaf. Esta capa nos permitirá aplicar lógica de negocio y manejar operaciones con anotaciones transaccionales como `@Transactional`.



1. Crear la Interface `PersonaService`

Ubicación: `src/main/java/mx/com/gm/servicio/PersonaService.java`

Creamos una nueva interfaz que definirá las operaciones del servicio:

```
package gm.servicio;

import java.util.List;
import gm.domain.Persona;

public interface PersonaService {

    List<Persona> listarPersonas();

    void guardar(Persona persona);
}
```

```
void eliminar(Persona persona);

Persona encontrarPersona(Persona persona);
}
```

Explicación:

- Se definen 4 métodos para el manejo CRUD de personas.
- Se importa la clase `Persona` desde el paquete `domain`.



2. Crear la Implementación `PersonaServiceImpl`

Ubicación: `src/main/java/mx/com/gm/servicio/PersonaServiceImpl.java`

```
package gm.servicio;

import java.util.List;
import gm.dao.PersonaDao;
import gm.domain.Persona;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class PersonaServiceImpl implements PersonaService {

    @Autowired
    private PersonaDao personaDao;

    @Override
    @Transactional(readOnly = true)
    public List<Persona> listarPersonas() {
        return (List<Persona>) personaDao.findAll();
    }

    @Override
    @Transactional
    public void guardar(Persona persona) {
        personaDao.save(persona);
    }

    @Override
    @Transactional
```

```

public void eliminar(Persona persona) {
    personaDao.delete(persona);
}

@Override
@Transactional(readonly = true)
public Persona encontrarPersona(Persona persona) {
    return personaDao.findById(persona.getIdPersona()).orElse(null);
}
}

```

Puntos Clave:

- `@Service` marca la clase como parte del contenedor de Spring.
- `@Transactional(readonly = true)` se usa para métodos que solo consultan datos.
- Métodos guardar y eliminar modifican datos, por eso solo se usa `@Transactional`.
- Se puede marcar toda la clase con la anotación `@Transactional` para hacer todos los métodos transaccionales, y sólo agregar `@Transactional(readonly = true)` para los métodos que solo consultan datos.



3. Inyectar el Servicio en el Controlador

Ubicación: `src/main/java/mx/com/gm/web/ControladorInicio.java`

Reemplaza la inyección del `PersonaDao` por el servicio `PersonaService`:

```

package gm.web;

import gm.servicio.PersonaService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ControladorInicio {

    private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

    @Autowired
    private PersonaService personaService;

```

```
@GetMapping("/")
public String inicio(Model model) {
    log.info("Ejecutando el controlador Spring MVC");
    var personas = personaService.listarPersonas();
    model.addAttribute("personas", personas);
    return "index";
}
```

Paso 4: Ejecutar y probar

 Ejecuta la clase `HolaSpringApplication.java` ubicada en:

`src/main/java/gm/HolaSpringApplication.java`

 Luego ve a tu navegador y abre:

<http://localhost:8080>

 Deberías ver una tabla con las personas registradas en tu base de datos.

 El resultado es el mismo de la lección pasada, pero ahora ya se agregó la capa de servicio a nuestra arquitectura con Spring.

Conclusión

✨ Ahora nuestra arquitectura está completa:

- **Controlador** maneja la presentación.
- **Servicio** maneja la lógica y delega al DAO.
- **DAO** accede directamente a la base de datos.

El flujo está desacoplado y profesional. Este diseño permite escalar la aplicación fácilmente.

¡Excelente trabajo!

Ya tienes una arquitectura profesional con manejo transaccional, servicios desacoplados y buenas prácticas de desarrollo en Spring Boot  

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)